

Challenge: Autonomous Maze Navigation using Fuzzy Logic Inference

1. Overview

Your objective is to design and implement a Fuzzy Logic Inference System (FIS) in MATLAB to successfully navigate a differential-drive robot through an unknown, randomly generated 2D maze.

You are provided with a high-fidelity MATLAB simulation environment (`mazeSim.m`) equipped with a real-time UI, ray-cast proximity sensors, and realistic differential-drive kinematics. Your task is to write the brain of the robot (`myController.m`) utilizing a `.fis` model to guide the robot from the starting position to the goal without hitting the walls.

2. System Specifications & Constraints

2.1 The Robot (Differential Drive)

The robot is driven by two independent wheels. Its motion is determined by the speed of the left motor (v_L) and the right motor (v_R).

- **Max Forward Speed:** 2.8 m/s per wheel.
- **Max Reverse Speed:** -2.8 m/s per wheel.
- **Turning:**
 - Turn Right: $v_L > v_R$
 - Turn Left: $v_R > v_L$
 - Spin in Place: $v_L = -v_R$

2.2 The Sensors

The robot is equipped with three LiDAR-style ray-cast proximity sensors measuring the distance to the nearest wall.

- **Sensor Layout:** `sensors.front`, `sensors.left`, `sensors.right`.
- **Minimum Range:** 0.0 m (Collision).
- **Maximum Range:** 5.0 m (If no wall is detected within 5 meters, the sensor outputs 5.0).
- **Critical Proximity:** Walls closer than 0.6 m front or 0.4 m sides require immediate evasion.

3. Challenge Requirements

3.1 Fuzzy Logic Design

You must design your controller using the Fuzzy Logic Toolbox in MATLAB (via the fuzzyLogicDesigner app or programmatic generation).

Step 1: Fuzzification (Inputs)

Define exactly 3 input variables corresponding to the three sensors.

- Range: [0, 5.0] meters.
- Membership Functions: Define appropriate linguistic terms (e.g., TooClose, Safe, Clear).

Step 2: Rule Base

Formulate a robust set of IF-THEN rules to handle obstacle avoidance and wall-following (if necessary).

- Example: IF (Front is TooClose) AND (Left is Safe) THEN (vL is Fast) AND (vR is Reverse)

Step 3: Defuzzification (Outputs)

Define exactly 2 output variables for the wheel speeds.

- Range: [-2.8, 2.8] m/s.
- Membership Functions: Define terms like Reverse, Stop, SlowForward, FastForward.

3.2 Integration

You must save your designed fuzzy system as a .fis file (e.g., navigator.fis) and evaluate it inside the provided myController.m script.

To ensure performance and prevent reloading the file every frame, use the persistent keyword in your controller. Your myController.m should look similar to this structure:

```
function [vL, vR] = myController(state, sensors)
% myController - Fuzzy Logic Navigation

    % Load the FIS file only once into memory
    persistent fis
    if isempty(fis)
        fis = readfis('navigator.fis');
    end

    % Stop moving if the goal is reached
    if state.goalReached
        vL = 0; vR = 0;
        return;
    end

    % 1. Gather inputs
    inputs = [sensors.left, sensors.front, sensors.right];

    % 2. Evaluate Fuzzy Inference System
    outputs = evalfis(fis, inputs);
```

```
% 3. Extract wheel speeds
vL = outputs(1);
vR = outputs(2);
end
```

4. Deliverables

Submit a single .zip file containing:

1. 1. navigator.fis (Your configured fuzzy logic model).
2. 2. myController.m (Your integration script).
3. 3. A short PDF report (1-2 pages) containing:
 - • Screenshots of your Input/Output Membership Functions.
 - • A screenshot of your Rule Viewer surface plot.
 - • A brief explanation of your logic strategy (e.g., "Left-wall follower", "Pure obstacle avoidance").